Mathematical Proofs, Shape Transitions, and Structural Flows of Low-Rank Adaptation (LoRA)

Chief Strategist J

June 12, 2026

1 Introduction and Intrinsic Dimension Hypothesis

Fine-tuning a large causal language model like Qwen2.5-0.5B requires optimizing high-dimensional parameter spaces. For a linear layer parameter matrix $W_0 \in \mathbb{R}^{d \times k}$, full parameter updates can be represented as:

$$W = W_0 + \Delta W \tag{1}$$

Computing and storing updates ΔW directly requires maintaining optimizer states of size $O(d \cdot k)$ parameters per layer. Low-Rank Adaptation (LoRA) operates on the hypothesis that the update matrix ΔW resides in a subspace with a very low intrinsic dimension or rank $r \ll \min(d, k)$. Consequently, we define the update matrix by factorizing it:

$$\Delta W = \frac{\alpha}{r} BA \tag{2}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable, while the original W_0 is frozen.

2 Index-Level Derivation of Backpropagation

Let $x \in \mathbb{R}^{1 \times d}$ be the row-vector input activation. The output projection $h \in \mathbb{R}^{1 \times k}$ is:

$$h_j = \sum_{i=1}^{d} x_i (W_0)_{ij} + \frac{\alpha}{r} \sum_{p=1}^{r} \left(\sum_{i=1}^{d} x_i B_{ip} \right) A_{pj} \tag{3}$$

Let $\mathcal{L}$ be the scalar cross-entropy loss. We define the incoming gradient with respect to output h as:

$$g_j = \frac{\partial \mathcal{L}}{\partial h_j} \tag{4}$$

2.1 Gradient with respect to Matrix A

To update the matrix $A \in \mathbb{R}^{r \times k}$, we take the partial derivative of $\mathcal{L}$ with respect to a single element A_{qj} (where $q \in \{1, \dots, r\}$ and $j \in \{1, \dots, k\}$):

$$\frac{\partial \mathcal{L}}{\partial A_{qj}} = \sum_{m=1}^{k} \frac{\partial \mathcal{L}}{\partial h_m} \frac{\partial h_m}{\partial A_{qj}} \tag{5}$$

Since h_m depends on A_{qj} only when $m = j$, the summation collapses:

$$\frac{\partial \mathcal{L}}{\partial A_{qj}} = \frac{\partial \mathcal{L}}{\partial h_j} \frac{\partial h_j}{\partial A_{qj}} = g_j \cdot \frac{\partial}{\partial A_{qj}} \left[\sum_{i=1}^{d} x_i (W_0)_{ij} + \frac{\alpha}{r} \sum_{p=1}^{r} \left(\sum_{i=1}^{d} x_i B_{ip} \right) A_{pj} \right] \tag{6}$$

Taking the derivative with respect to A_{qj} isolates the terms where $p = q$:

$$\frac{\partial h_j}{\partial A_{qj}} = \frac{\alpha}{r} \sum_{i=1}^{d} x_i B_{iq} = \frac{\alpha}{r} (xB)_q \tag{7}$$

Substituting this back, we obtain the element-wise gradient:

$$\frac{\partial \mathcal{L}}{\partial A_{qj}} = \frac{\alpha}{r} (xB)_q \cdot g_j \tag{8}$$

Expressed in matrix notation, this corresponds to the outer product of the bottleneck activations and the gradient vector:

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\alpha}{r} (xB)^T g \tag{9}$$

2.2 Gradient with respect to Matrix B

Now, we compute the partial derivative of $\mathcal{L}$ with respect to a single element B_{iq} (where $i \in \{1, \dots, d\}$ and $q \in \{1, \dots, r\}$):

$$\frac{\partial \mathcal{L}}{\partial B_{iq}} = \sum_{j=1}^{k} \frac{\partial \mathcal{L}}{\partial h_j} \frac{\partial h_j}{\partial B_{iq}} = \sum_{j=1}^{k} g_j \cdot \frac{\partial h_j}{\partial B_{iq}} \tag{10}$$

Looking at the forward equation, the variable B_{iq} only appears in the term where $p = q$:

$$\frac{\partial h_j}{\partial B_{iq}} = \frac{\partial}{\partial B_{iq}} \left[\frac{\alpha}{r} \left(\sum_{l=1}^{d} x_l B_{lq} \right) A_{qj} \right] = \frac{\alpha}{r} x_i A_{qj} \tag{11}$$

Substituting this derivative back into the sum yields:

$$\frac{\partial \mathcal{L}}{\partial B_{iq}} = \sum_{j=1}^{k} g_j \left(\frac{\alpha}{r} x_i A_{qj} \right) = \frac{\alpha}{r} x_i \left(\sum_{j=1}^{k} g_j A_{qj} \right) = \frac{\alpha}{r} x_i (gA^T)_q \tag{12}$$

In matrix notation, this is represented by:

$$\frac{\partial \mathcal{L}}{\partial B} = \frac{\alpha}{r} x^T (gA^T) \tag{13}$$

3 Neuron-Level Activation and Gradient Flow Diagram

The diagram below illustrates a detailed neuron-level visualization of the Low-Rank Adaptation. The input vector x feeds into both the massive frozen base weights W_0 and the trainable low-rank pathway consisting of projection matrix B to a bottleneck activation layer z of size r, which is projected back via matrix A to the size k, scaled, and summed.

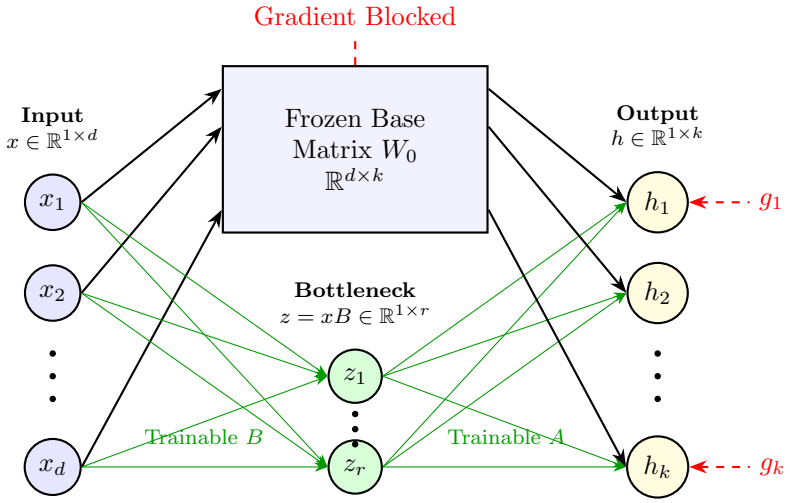

Figure 1: Neuron-level forward path (solid arrows) and backward gradient routing (dashed red arrows).

4 Functional Pipelines: Before State, After State, and Flow Control

We present the complete code implementation, before and after states, and dedicated transformation flow diagrams for each helper function.

4.1 Model and Tokenizer Loader: `initialize_base_model`

4.1.1 Functional Code Implementation

The initializer handles loading vocabulary parameters and floating-point representations of the model weights from the local cache directory. Memory bounds are defined based on precision casting.

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

def initialize_base_model(model_id="Qwen/Qwen2.5-0.5B-Instruct"):
    tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
    tokenizer.pad_token = tokenizer.eos_token

    # Setup hardware parameters automatically based on GPU compatibility
    torch_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
    device_map = "auto" if torch.cuda.is_available() else "cpu"

    model = AutoModelForCausalLM.from_pretrained(
        model_id,
        torch_dtype=torch_dtype,
        device_map=device_map,
        trust_remote_code=True
    )
    return model, tokenizer
```

4.1.2 Parameter States Table

Property	Before State	After State
RAM Allocation	0 MB	$\approx$ 1024 MB
Loaded Variables	None	Model (CausalLM) + Tokenizer
Weight Matrix Shape W_0	Unloaded	$\mathbb{R}^{896\times896}$
Embedding Matrix Shape W_{embed}	Unloaded	$\mathbb{R}^{151936\times896}$
Grad Requirements	None	requires_grad = True

Table 1: State properties and shapes for model initialization.

4.1.3 Logical Transformation Flow (`initialize_base_model`)

This diagram visualizes weight file retrieval from disk storage, decompression/casting in the loading stream, and allocations into system memory.

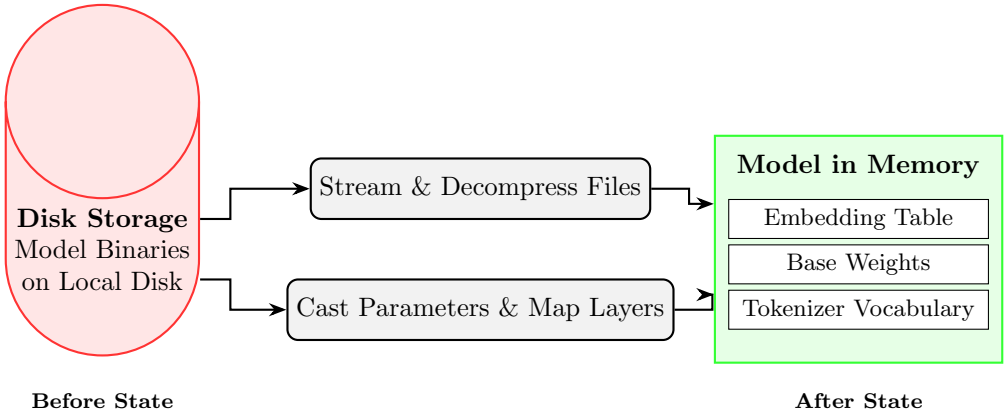

Figure 2: System load transformation mapping disk-based weights to RAM tensors.

4.2 PEFT Wrapper Layer Adapter: `apply_lora_adapters`

4.2.1 Functional Code Implementation

Adapter parameters are structured to map dimensions from activation scale d to constraint bottleneck rank r. Layer weights parameters are wrapped to reroute backpropagation pipelines.

```python
from peft import LoraConfig, get_peft_model, TaskType

def apply_lora_adapters(model, rank=8, alpha=16):
    peft_config = LoraConfig(
        task_type=TaskType.CAUSAL_LM,
        r=rank,
        lora_alpha=alpha,
        lora_dropout=0.05,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
        bias="none"
    )
    # Wraps the model and freezes base weight parameters automatically
    model = get_peft_model(model, peft_config)
    return model
```

4.2.2 Parameter States Table

Property	Before State	After State
Layer Wrapping	Plain CausalLM Linear	PEFT Wrapped Module
Base Weights W_0 Grads	requires_grad = True	requires_grad = False (Frozen)
Trainable Parameters	$\approx$ 500 Million	$\approx$ 6.2 Million (LoRA matrices)
LoRA Matrix B Shape	Does not exist	$\mathbb{R}^{896\times 8}$ (Initialized to 0)
LoRA Matrix A Shape	Does not exist	$\mathbb{R}^{8\times 896}$ (Initialized to Gaussian)

Table 2: State details before and after adapter injection.

4.2.3 Logical Transformation Flow (`apply_lora_adapters`)

This diagram maps the structural change of a linear mapping, shifting from a single active matrix to a frozen base weight matrix bypassed by trainable low-rank path networks.

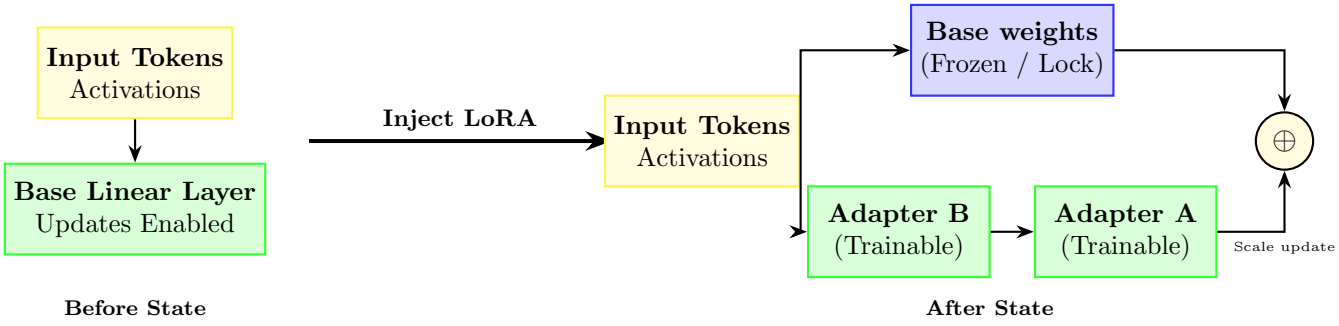

Figure 3: Parameter transformation flow mapping adapter logic injection.

4.3 Tokenization and Label Masking: `tokenize_and_mask_labels`

4.3.1 Functional Code Implementation

Preprocesses raw template sequences, formatting system instructions and isolating debugger output strings for loss computation mapping.

```
def tokenize_and_mask_labels(examples, tokenizer):
    prompts = [f"<|im_start|>system\nYou are a formatting engine for pylow. Output ASCII layout
    layout correctly.<|im_end|>\n<|im_start|>user\n{inp}<|im_end|>\n<|im_start|>assistant\n" for inp
     in examples["input"]]
    targets = [f"{out}<|im_end|>" for out in examples["output"]]

    inputs = tokenizer(prompts, add_special_tokens=False)
    labels = tokenizer(targets, add_special_tokens=False)

    input_ids = []
    label_ids = []
    for i in range(len(prompts)):
        # Concatenate prompt inputs and output targets
        input_ids.append(inputs["input_ids"][i] + labels["input_ids"][i])
        # Mask out system prompt sequence with -100
        label_ids.append([-100] * len(inputs["input_ids"][i]) + labels["input_ids"][i])

    return {"input_ids": input_ids, "labels": label_ids}
```

4.3.2 Parameter States Table

Property	Before State	After State
Format Type	Raw Python Strings	Integer Tensors
Input IDs Shape	1 (Scalar String)	$\mathbb{Z}^{\text{seq\_len}}$
Label IDs Shape	1 (Scalar String)	$\mathbb{Z}^{\text{seq\_len}}$
Prompt Labels Value	Raw string representation	Masked out with -100
Response Labels Value	Raw string representation	Int token identifiers preserved

Table 3: Data structures before and after tokenization.

4.3.3 Logical Transformation Flow (`tokenize_and_mask_labels`)

This visual mapping details how text strings translate to aligned vocabulary integer indices, showing exactly how prompt loss calculation parameters are masked out.

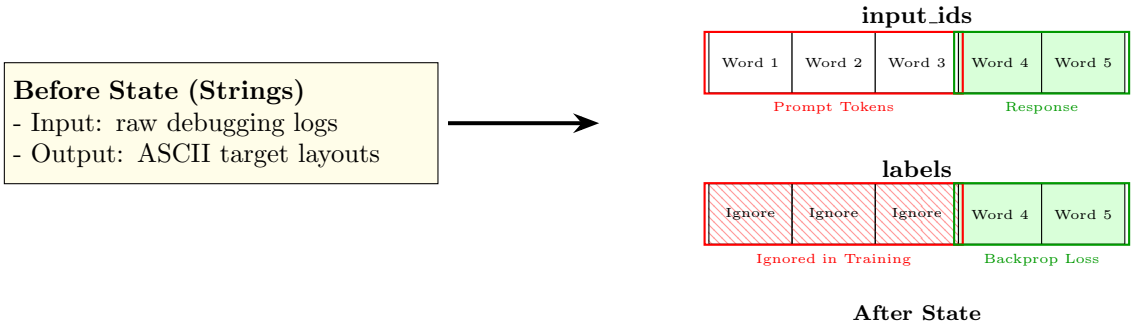

Figure 4: Parameter transformation flow mapping string sequences to masked label tokens.

4.4 Training Execution Loop: `execute_training_loop`

4.4.1 Functional Code Implementation

The training execution block establishes micro-batch iterations, gradient scaling accumulation limits, and logs parameter updates to disk files.

```
from transformers import TrainingArguments, Trainer, DataCollatorForSeq2Seq

def execute_training_loop(model, tokenizer, tokenized_dataset, output_path):
    training_args = TrainingArguments(
        output_dir=output_path,
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        logging_steps=10,
        num_train_epochs=3,
        save_strategy="epoch",
        evaluation_strategy="no",
        fp16=False,
        bf16=False,
        optim="adamw_torch"
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_dataset,
        data_collator=DataCollatorForSeq2Seq(tokenizer=tokenizer, pad_to_multiple_of=8,
    return_tensors="pt", padding=True)
    )
    trainer.train()
    model.save_pretrained(f"{output_path}/adapter")
```

4.4.2 Parameter States Table

Property	Before State	After State
Adapter Parameters	$A \sim \mathcal{N}(0, \sigma^2)$, $B = 0$	Optimized weights A^*, B^*
Causal LM Loss	$\mathcal{L} \approx 10.0$	$\mathcal{L} < 0.1$
Active Memory Footprint	1024 MB (weights)	1024 MB + Optimizer states
Checkpoints	Empty directory	Adapter weights saved on disk

Table 4: Parameter states and loss during the training phase.

4.4.3 Logical Transformation Flow (`execute_training_loop`)

The diagram below details the forward-backward optimization loop that updates the trainable adapter parameters.

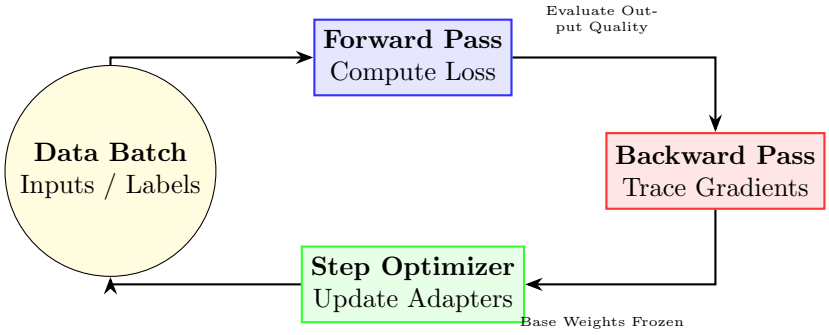

Figure 5: Circular flow mapping of the training step execution loop.

4.5 Zero-Overhead Deployment Merge: `merge_adapters_for_deployment`

4.5.1 Functional Code Implementation

Resolves model output path speed penalties by collapsing trainable parameters back into fixed weights.

```python
from peft import PeftModel

def merge_adapters_for_deployment(base_model_id, adapter_path, merged_output_path):
    base_model = AutoModelForCausalLM.from_pretrained(
        base_model_id,
        torch_dtype=torch.float16,
        device_map="cpu"
    )
    # Attach adapter parameters
    peft_model = PeftModel.from_pretrained(base_model, adapter_path)

    # Merge BA into W_0 and unload PEFT packaging
    merged_model = peft_model.merge_and_unload()

    # Serialize to disk
    merged_model.save_pretrained(merged_output_path)
    print("Merged weights saved successfully!")
```

4.5.2 Parameter States Table

Property	Before State	After State
PEFT Wrapping	Wrapper active in memory	Removed
Active Parameter Sets	$\{W_0, B, A\}$ (Three matrices)	$\{W_{\text{deploy}}\}$ (Single matrix)
Layer Output Comp.	$h = xW_0 + \frac{\alpha}{r}xBA$	$h = xW_{\text{deploy}}$
Inference Overhead	Extra matrix multiplication	Zero extra latency
Weights File shape	Base + Adapter binaries	Unified model weights binary

Table 5: Parameter representations during adapter merging.

4.5.3 Logical Transformation Flow (`merge_adapters_for_deployment`)

This diagram maps parameter addition, merging parallel layers into a single weight tensor.

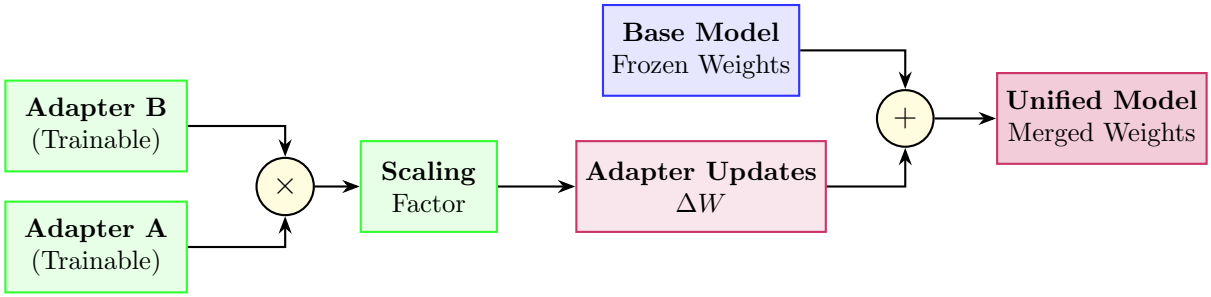

Figure 6: Consolidation flow of multiple weights paths into a single deployment matrix.